Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar

A
PROJECT REPORT
On

"Customer Behavior Data Analysis"

For the degree of
BACHELOR OF COMPUTER SCIENCE

Submitted by

Mr. Irfan Rubab Pathan

&

Mr. Vitthal Bhagwan Munde

Under The Guidance Of
Shahbaz Shaikh
Ass. Professor

Through

SBSPM'S
College of Computer Science & IT , Ambajogai

Academic Year
[2025-2026]

Dr. Babasaheb Ambedkar Marathwada University, Chh. Sambhajinagar
College of Computer Science & IT , Ambajogai

CERTIFICATE

This is to certify that a major project report on **"Customer Behavior Data Analysis"** it has been successfully completed and submitted by **Mr. Irfan Rubab Pathan** as requirement of **Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar** in partial fulfilment of BCS Degree (Bachelor of Computer Science)-Final Year for the academic year 2025-2026.

Project Guide **Principal**

External Examiner

Dr. Babasaheb Ambedkar Marathwada University, Chh. Sambhajinagar
College of Computer Science & IT , Ambajogai

CERTIFICATE

This is to certify that a major project report on **"Customer Behavior Data Analysis"** it has been successfully completed and submitted by **Mr. Vitthal Bhagwan Munde** as requirement of **Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar** in partial fulfilment of BCS Degree (Bachelor of Computer Science)-Final Year for the academic year 2025-2026.

Project Guide **Principal**

External Examiner

Acknowledgement

We are deeply grateful to our guide **Shahbaz Shaikh** sir for providing wholehearted guidance and invaluable support throughout the completion of our major project. The consistent encouragement and constructive feedback from our guide helped us navigate the technical challenges and deliver a well-structured final product.

We would like to express our sincere appreciation to the Principal **Dr. T C Gupta** Maam and the entire faculty of the College of Computer Science & Information Technology Ambajogai, for their unfailing cooperation, continuous motivation, and timely guidance & creating an academic environment that nurtures innovation and practical application of knowledge.

Special thanks to our friends for their moral support and patience throughout the project development process. Their belief in our abilities gave us the confidence to complete this work successfully.

Irfan Pathan & Vitthal Munde

BCS (Bachelor of Computer Science)

College of Computer Science & IT, Ambajogai

DECLARATION

We, **Mr. Irfan Rubab Pathan & Mr. Vitthal Bhagwan Munde**, students of BCS (Bachelor of Computer Science) Final Year, College of Computer Science & IT, Ambajogai, hereby declare that the project report entitled **"Customer Behavior Data Analysis"** submitted to Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar, in partial fulfilment of the requirements for the degree of Bachelor of Computer Science, is a record of original work done by us under the guidance of Shahbaz Shaikh, Ass. Professor, College of Computer Science & IT, Ambajogai.

We further declare that this project report has not been previously submitted for the award of any degree, diploma, or any other academic distinction of this or any other university.

Place: Signature

Date:

INDEX

1. Introduction:

I am pleased to introduce the **Customer Behavior Data Analytics System**, a project tailored to meet the sophisticated data needs of the modern retail industry and specifically developed for the academic community at **Dr. Babasaheb Ambedkar Marathwada University, Chh. Sambhaji Nagar**. This system serves as a powerful analytical engine that transforms raw consumer data into clear, actionable insights through a structured end-to-end workflow. By analyzing factors such as purchasing patterns, demographics, and payment preferences, the project offers a comprehensive view of how consumers interact with a brand. Moreover, it provides data-driven recommendations on marketing strategies and product optimization to ensure long-term customer loyalty. With its interactive Power BI dashboard and rigorous SQL-backed analysis, the system ensures that complex data is accessible and easy to understand for decision-makers.

The Customer Behavior Data Analytics System is a strategic tool that bridges the gap between raw numbers and business growth. Using a combination of Python for data cleaning, PostgreSQL for deep querying, and Power BI for visualization, it allows users to identify trends and optimize performance in real-time. It is designed to be user-friendly, providing stakeholders with reliable metrics to take control of business outcomes in a simple yet effective way.

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

1.1. Objectives:

The primary objective of this project is to execute a complete, company-level data analytics lifecycle on retail data. Specific goals include:

1. Analyzing customer shopping behavior to understand purchasing patterns across demographics and categories.
2. Identifying the factors driving consumer decisions, such as discounts, shipping types, and subscription statuses.
3. Developing an interactive Power BI dashboard to present key performance indicators (KPIs) to business stakeholders for optimizing marketing and product strategies.
4. End-to-End Workflow: To implement a full data analytics lifecycle from raw data to business presentation.
5. Technical Integration: To demonstrate the seamless flow of data between Python (Cleaning), SQL (Storage/Querying), and Power BI (Visualization).
6. Strategic Growth: To provide the retail company with data-backed recommendations for marketing and inventory management.

1.2. Business Problem Statement:

A leading retail company wants to better understand its customers' shopping behavior in order to improve sales, customer satisfaction, and long-term loyalty. The management team has noticed changes in purchasing patterns across demographics, product categories, and sales channels (online vs. offline). They are particularly interested in uncovering which factors, such as discounts, reviews, seasons, or payment preferences, drive consumer decisions and repeat purchases. You are tasked with analyzing the company's consumer behavior dataset to answer the following overarching business question: "How can the company leverage consumer shopping data to identify trends, improve customer engagement, and optimize marketing and product strategies?"

1.3. Hardware And Software Components:

- Hardware Requirements:

A standard PC or laptop with a minimum:

Processor: Intel i5 or higher (for handling local SQL server and Power BI).

RAM: 8GB minimum (16GB recommended for Running Smoothly Power BI desktop).

Storage: 256GB SSD for local database hosting.

- Software Requirements:

1. Operating System: Above or Windows 10 (Recommended).
2. Python Environment: Jupyter Notebook.
3. PostgreSQL / pgAdmin 4: For Relational Database Management.
4. Power BI Desktop: For Interactive Dashboarding.
5. Web Browser: Google Chrome or equivalent.
6. Internet Connection: Required for downloading dependencies, libraries and all the softwares.

1.4. Literature survey:

Modern retail analytics has shifted from simple "Total Sales" reporting to "Customer-Centric" analysis. Previous studies show that retaining a loyal customer is 5x cheaper than acquiring a new one. This project builds on these findings by focusing on Customer Segmentation and Discount Sensitivity, which are the current industry standards for increasing Profit Margin and Customer Lifetime Value (CLV).

In corporate retail environments, raw data is rarely analyzed directly in Excel due to scale and security constraints. Modern data analytics relies heavily on SQL databases and Business Intelligence (BI) tools.

This project aligns with industry standards by demonstrating that effective analysis requires structured data cleaning (e.g., handling missing values intelligently rather than blindly) and robust querying to uncover actionable business metrics, such as evaluating whether discount strategies genuinely drive higher spending or if subscription models yield higher total revenue.

2. About Data Analytics & How it is used in Project:

Data Analytics is the process of examining data sets to find trends and draw conclusions about the information they contain. In a corporate environment, it acts like a "business compass" that helps companies understand where they are and where they need to go. It uses special technology to process thousands of rows of information and turn them into charts and reports that make sense**.**

There are primarily fmy types of Analytics used in this project:

1. **Descriptive Analytics (The "What happened?" phase):**

- This summarizes past data to understand what has occurred in the business.
- It follows a structured flow of looking at historical sales and customer counts.
- Example: Using Power BI to show that sales increased by 10% last month.

2. **Diagnostic Analytics (The "Why did it happen?" phase):**

- This leverages deeper data exploration to find the root cause of trends.
- It continuously looks for correlations, such as whether a discount led to more sales.
- Example: Using SQL queries to find out if a drop in revenue was caused by a specific product category or shipping delay.

3. **Prescriptive Analytics (The "How can we make it happen?" phase):**

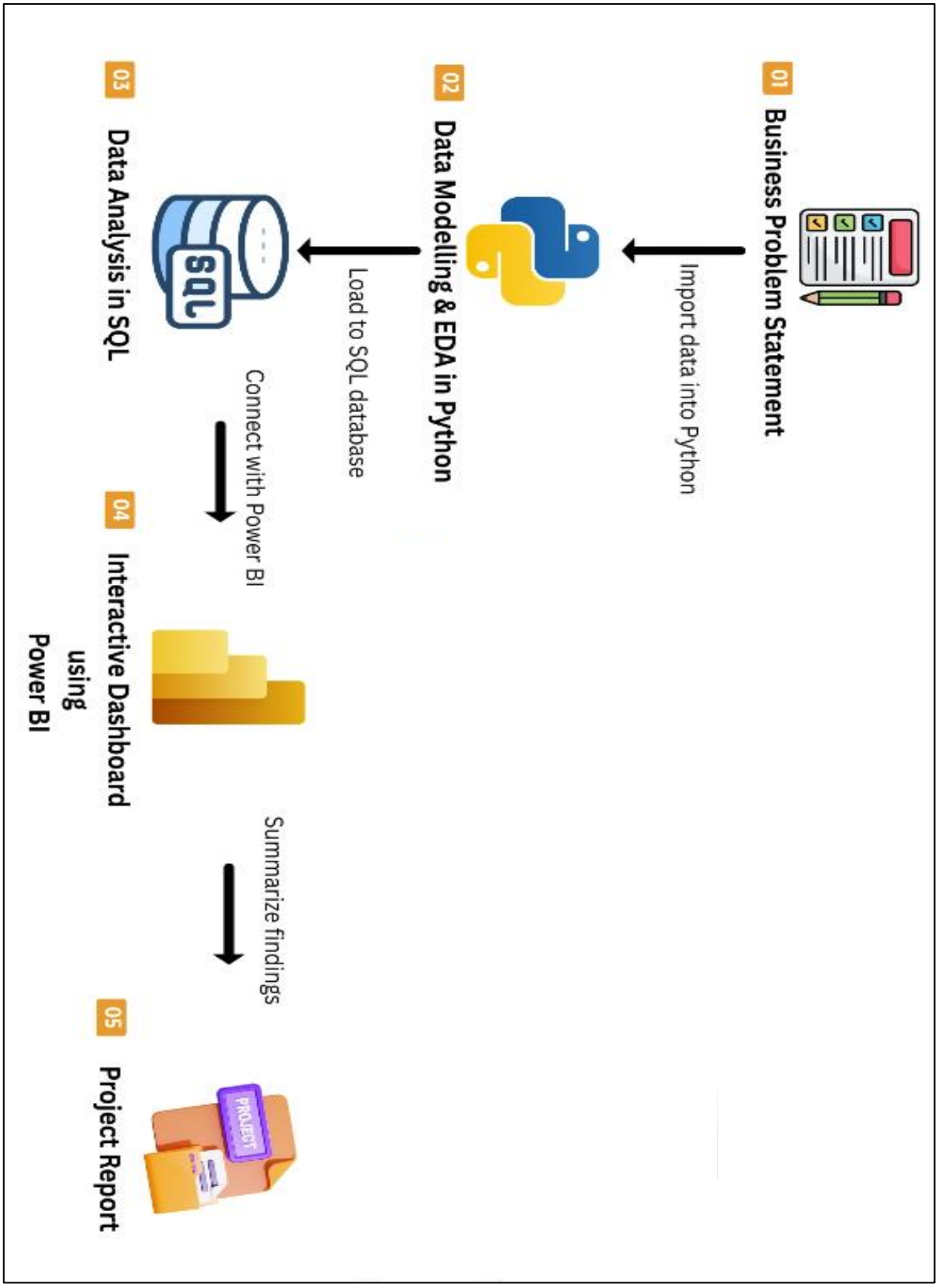
- This suggests specific actions to take based on the data findings.
- It offers the final strategy for marketing and product development.
- Example: Recommending a loyalty program for "High-Value" customers to increase repeat purchases.

3. Technologies used:

Python programming language**:** Python is a high-level programming language known for its simplicity and versatility. It's widely used in web development, data analysis, and artificial intelligence. Python's readability and extensive libraries make it ideal for beginners and professionals alike. Its cross-platform compatibility and active community support further enhance its popularity and usability. Overall, Python is a powerful tool for solving a variety of programming tasks efficiently.

1. **Data Processing:** Python (for Exploratory Data Analysis and Feature Engineering).
2. **Jupyter Notebook (The Interface):** It allows you to write python code, view dataframes, and create visualizations in "cells.
3. **Database Management:** SQL (for data storage, aggregation, and advanced analysis using window functions).
4. **Business Intelligence:** Power BI (for creating interactive, client-ready dashboards).

4. Project Workflow :



5. Algorithm:

The logical workflow of the data pipeline follows these sequential steps:

1. **Ingestion:** Load raw CSV data into a Pandas DataFrame.
2. **Imputation:** Handle missing review_rating values by calculating and applying the median rating specific to each product category, avoiding data bias.
3. **Transformation:** Standardize column names (snake_case) and engineer new features (e.g., mapping textual purchase frequencies to numeric days, categorizing ages into demographic bins).
4. **Database Migration:** Establish a connection via SQLAlchemy and load the cleaned DataFrame into a PostgreSQL table.
5. **Querying:** Execute SQL queries to extract insights (e.g., using CASE statements for customer segmentation and ROW_NUMBER() for ranking top products).
6. **Visualization:** Connect Power BI to the PostgreSQL database, create DAX measures, and build interactive visuals with slicers.

6. Libraries used in the Project:

- Core Python Libraries Used :

1) Pandas (pandas)

Purpose: This is the primary engine for data manipulation and exploratory data analysis (EDA).

Specific Use Cases in the Project: * Loading the raw data(customer_shopping_behavior.csv file).

Intelligently imputing missing review ratings using category-specific medians (df.groupby().transform()).

Feature engineering, such as creating demographic bins using quantiles (pd.qcut()) and mapping textual frequencies to numerical days.

Formatting column names to snake_case to prevent syntax errors later in SQL.

2) SQLAlchemy (sqlalchemy)

Purpose: A SQL toolkit and Object-Relational Mapper (ORM) that acts as the bridge between ymy Python environment and ymy database.

Specific Use Cases in the Project: Used specifically to generate the connection string with the create_engine() function, allowing the seamless insertion of the Pandas DataFrame directly into a PostgreSQL table using df.to_sql().

3) Psycopg2 (psycopg2)

Purpose: The most popular PostgreSQL database adapter for the Python programming language. It connects Jupyter Notebook with pgadmin4(PostgreSQL Interface).

Specific Use Cases in the Project: While not directly typed out line-by-line, it is the underlying engine that SQLAlchemy requires to communicate with PostgreSQL locally over port 5432.

7. Dataset:

A dataset is structured data compiled for analysis, often in table format, containing information on a particular subject. It serves as a resmyce for research, analysis, and modeling in various fields. Datasets facilitate understanding, exploration, and discovery through statistical analysis.
Smyce: Customer Shopping Behavior Dataset (CSV).
Structure: Each row represents a single customer's latest transaction.
Key Variables: Demographics (Age, Gender, Location), Transactional Data (Item, Category, Purchase Amount, Discount), and Behavioral Data (Review Rating, Frequency of Purchases, Previous Purchases).

➢ Dataset Summary :

- Rows: 3,900 - Columns: 18 - Key Features:
- Customer demographics (Age, Gender, Location, Subscription Status)
- Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
- Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of
- Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

➢ Images Of Dataset :

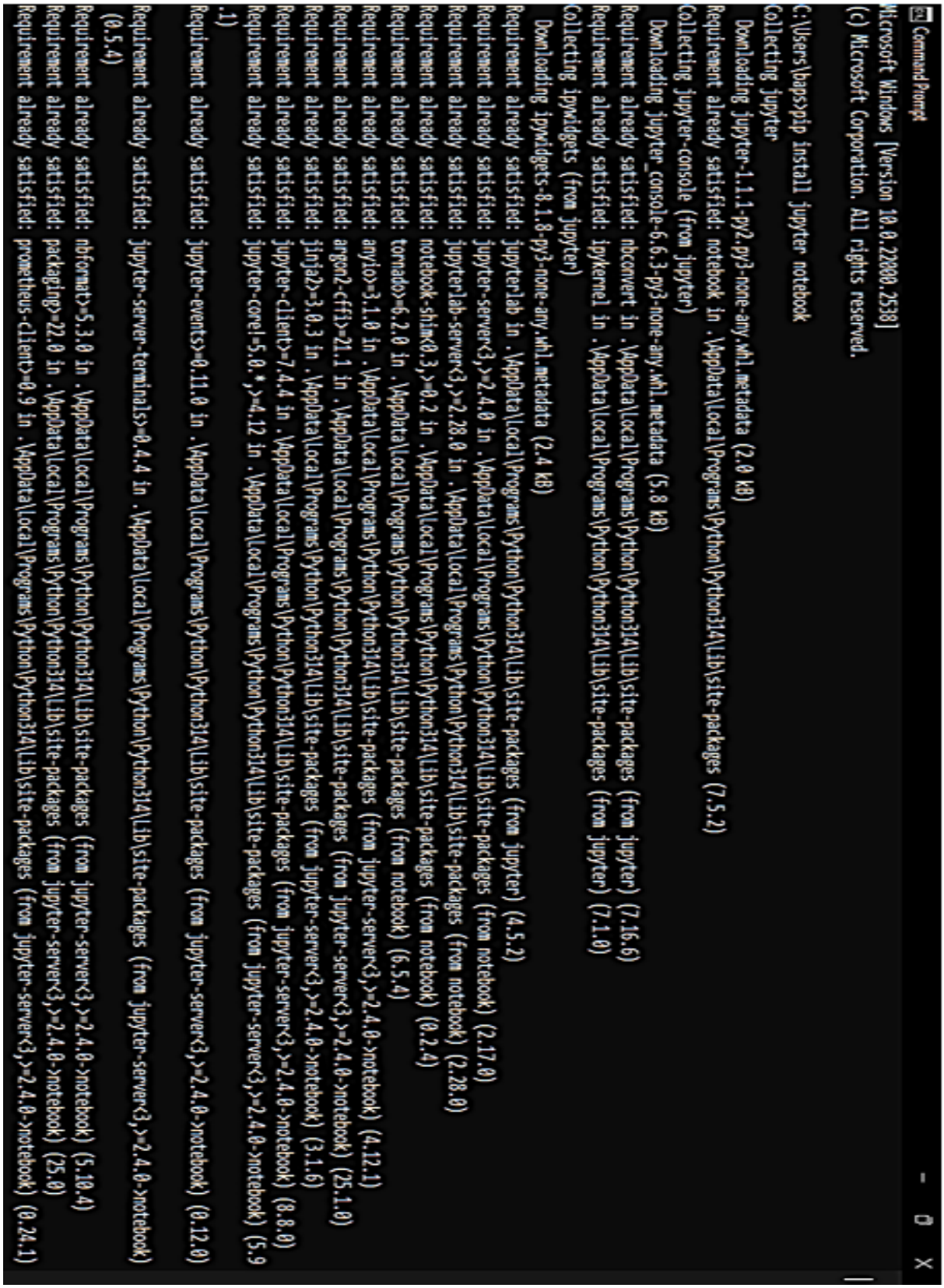
	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discou Appli
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	39
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	22
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	Na
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	Na
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	Na
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	Na
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	Na
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	Na
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	Na

Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
3900	3900	3900.000000	3900	3900
2	2	NaN	6	7
No	No	NaN	PayPal	Every 3 Months
2223	2223	NaN	677	584
NaN	NaN	25.351538	NaN	NaN
NaN	NaN	14.447125	NaN	NaN
NaN	NaN	1.000000	NaN	NaN
NaN	NaN	13.000000	NaN	NaN
NaN	NaN	25.000000	NaN	NaN
NaN	NaN	38.000000	NaN	NaN
NaN	NaN	50.000000	NaN	NaN

- **Missing Data Handling:** Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.
- **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.
- **Feature Engineering:**
    - Created **age_group** column by binning customer ages.
    - Created **purchase_frequency_days** column from purchase data.
- **Data Consistency Check:** Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.
- **Database Integration:** Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

8. Implementation Of Python Coding (Jupyter Notebook):

8.1) First we go in cmd(command prompt) and then type – pip install jupyter notebook(mine is already installed here so it is showing Requirement already satisfied).



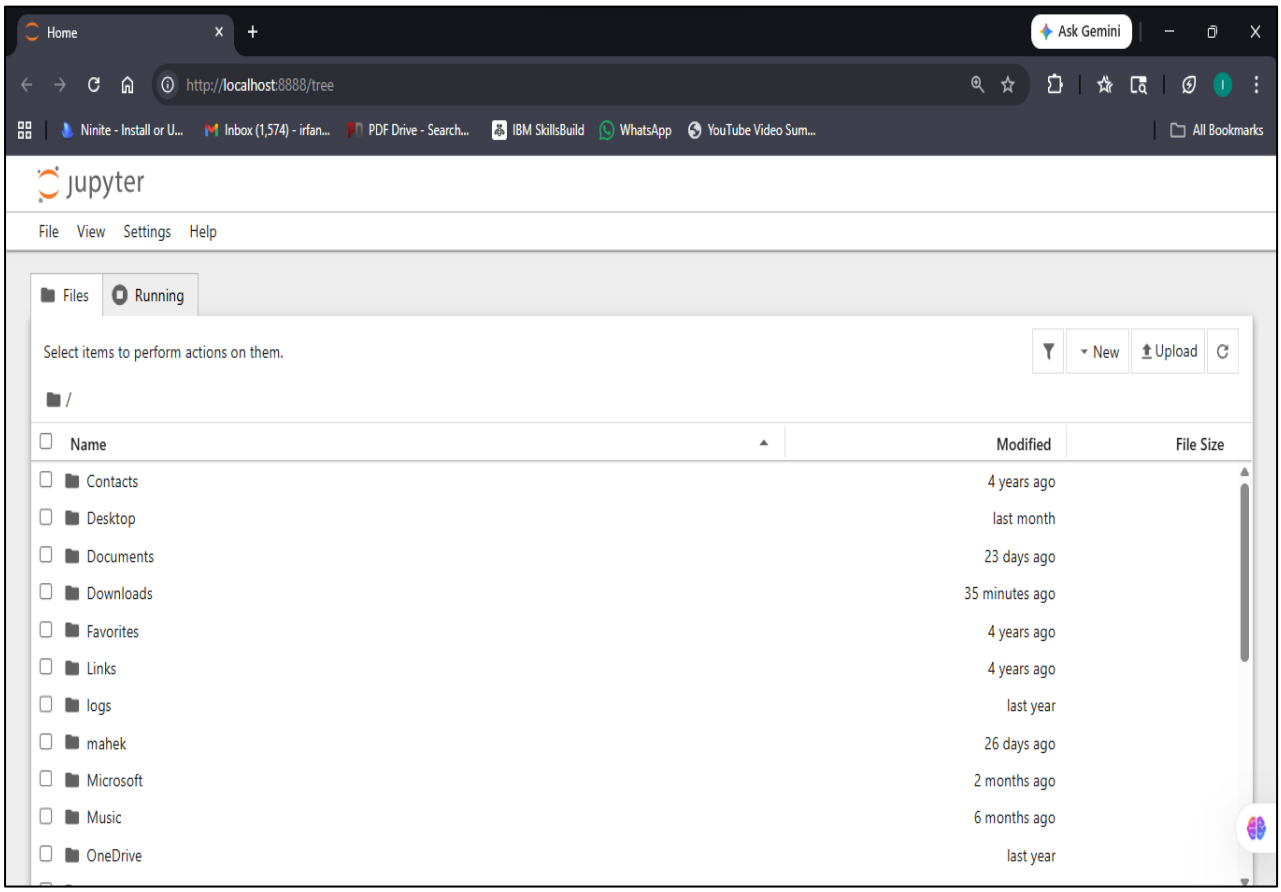
(Figure 8.1 Jupyter Notebook installation)

8.2) After that we type—jupyter notebook (to launch the application)

(Figure 8.2 Opening Jupyter Notebook)

8.3) Jupyter Notebook Interface:

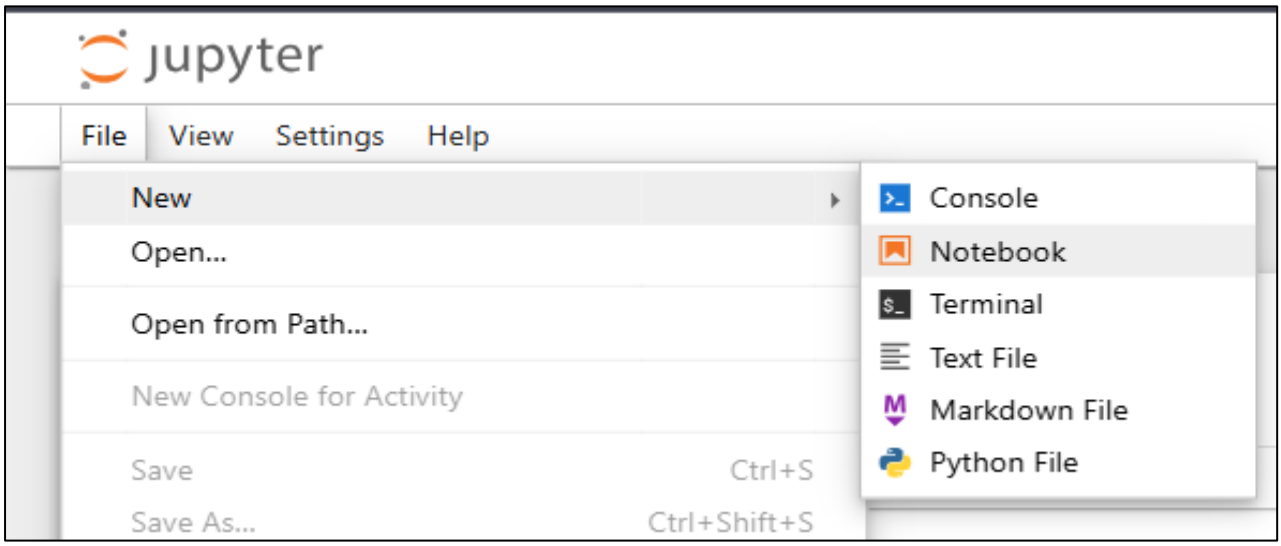
After that jupyter notebook automatically opens in default browser.



(Figure 8.3 Jupyter Notebook Interface)

8.4) New File Selection in Jupyter Notebook

Click to File→New→Notebook



(Figure 8.4 New File Selection)

8.5) Data Ingestion and Initial Inspection

The project begins by loading the raw dataset into the Python environment in Jupyter Notbebook using the Pandas library. This allows us to convert the flat .csv file into a structured DataFrame for manipulation.

```
[1]: import pandas as pd

     df= pd.read_csv('customer_shopping_behavior.csv')
```

(Figure 8.5 Loading Dataset and python library)

8.6) Initial Data Profiling and Exploration

df.head() is the first visual confirmation that the data was loaded correctly into the Pandas DataFrame. It displays the first 5 rows of raw dataset.

```
[2]: df.head()
```

[2]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	S
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	

(Figure 8.6 Loaded Dataset)

8.7) Data Audit and Null Value Detection

Using the df.info() method, we performed a structural audit of the dataset. This step is essential to understand the memory usage, data types (Dtypes), and completeness of the records. It proves that audited the data before jumping into analysis. the Review Rating column shows only 3863 non-null values. This indicates 37 missing entries (3900−3863=37).

```
[3]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   str
 3   Item Purchased          3900 non-null   str
 4   Category                3900 non-null   str
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   str
 7   Size                    3900 non-null   str
 8   Color                   3900 non-null   str
 9   Season                  3900 non-null   str
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   str
 12  Shipping Type           3900 non-null   str
 13  Discount Applied        3900 non-null   str
 14  Promo Code Used         3900 non-null   str
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   str
 17  Frequency of Purchases  3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

(Figure 8.7 Reviewing Dataset)

8.8) Exploratory Data Analysis: Statistical Profiling

After identifying the structural integrity of the data, we executed the df.describe() method to generate a statistical profile of the numerical features. This is a crucial step in Quality Assurance to ensure no "dirty data" (like negative ages or impossible prices) exists.

```
[5]: df.describe()
```

[5]:

	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3863.000000	3900.000000
mean	1950.500000	44.068462	59.764359	3.750065	25.351538
std	1125.977353	15.207589	23.685392	0.716983	14.447125
min	1.000000	18.000000	20.000000	2.500000	1.000000
25%	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1950.500000	44.000000	60.000000	3.800000	25.000000
75%	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3900.000000	70.000000	100.000000	5.000000	50.000000

(Figure 8.8 Performing EDA)

8.9) Categorical Profiling and Frequency Analysis

To gain a 360-degree view of the business, we utilized the include='all' parameter. This allows us to see statistics for non-numerical columns, specifically identifying the Unique counts and the Top (mode) performing categories.

```
[6]: df.describe(include='all')
```

[6]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN

(Figure 8.9 Useful Statistics of Dataset)

8.10) Missing Value Analysis

To verify the integrity of the dataset, we executed the df.isnull().sum() command. This provides a definitive count of missing values per feature. As confirmed in Figure 7, only the Review Rating column contains missing data, totaling 37 null entries.

```
[7]: df.isnull().sum()

[7]: Customer ID                0
     Age                        0
     Gender                     0
     Item Purchased             0
     Category                   0
     Purchase Amount (USD)      0
     Location                   0
     Size                       0
     Color                      0
     Season                     0
     Review Rating             37
     Subscription Status        0
     Shipping Type              0
     Discount Applied           0
     Promo Code Used            0
     Previous Purchases         0
     Payment Method             0
     Frequency of Purchases     0
     dtype: int64

[10]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median(
```

(Figure 8.10 Null Values Finding)

8.11) Verification of Data Cleansing

this section demonstrates that it followed a Test-Driven Development (TDD) approach by verifying that the null values were actually removed. After implementing the Category-Specific Median Imputation, it is standard professional practice to verify the results. We re-executed the df.isnull().sum() command to ensure the "Review Rating" column was successfully patched.

```
[10]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

[11]: df.isnull().sum()

[11]: Customer ID              0
      Age                      0
      Gender                   0
      Item Purchased           0
      Category                 0
      Purchase Amount (USD)    0
      Location                 0
      Size                     0
      Color                    0
      Season                   0
      Review Rating            0
      Subscription Status      0
      Shipping Type            0
      Discount Applied         0
      Promo Code Used          0
      Previous Purchases       0
      Payment Method           0
      Frequency of Purchases   0
      dtype: int64
```

(Figure 8.11 Cleaning Dataset)

8.12) Database-Ready Schema Conversion (Snake Casing)

A fundamental step in making a dataset "Production-Ready." For report, this illustrates the transition from raw, messy data to a clean, standardized format compatible with professional databases like PostgreSQL. we applied Snake Casing—the industry standard for database naming conventions.

```
[14]: df.columns = df.columns.str.lower()
      df.columns = df.columns.str.replace(' ', '_')
      df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})

[15]: df.columns

[15]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
             'purchase_amount', 'location', 'size', 'color', 'season',
             'review_rating', 'subscription_status', 'shipping_type',
             'discount_applied', 'promo_code_used', 'previous_purchases',
             'payment_method', 'frequency_of_purchases'],
            dtype='str')
```

(Figure 8.12 Applying Snake Casing)

8.13) Demographic Segmentation: Age Group

To better understand the purchasing power across different life stages, we implemented Binning. Analyzing customers by individual years, however, grouping them into broad categories allows the marketing team to design targeted campaigns (e.g., "Senior Discounts" or "Student Offers").

[16]:
```
# create a column age_group

labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']

df['age_group'] = pd.qcut (df['age'], q=4, labels = labels)


df[['age', 'age_group']].head(10)
```

[16]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

(Figure 8.13 Analysing Customers By Age Categories)

8.14) Feature Engineering: Purchase Frequency Quantization

To enable numerical analysis of customer habits, we transformed the frequency_of_purchases column. While labels like "Weekly" or "Fortnightly" are useful for human reading, they cannot be used for calculating averages or correlations in their raw string format.

We created a Python dictionary, frequency_mapping, to assign a discrete number of days to each textual label. We then utilized the .map() function to generate a new feature: purchase_frequency_days. this section is vital because it shows your ability to prepare data for mathematical modeling and advanced SQL aggregations.

```
[18]: # create column purchase_frequency_days

frequency_mapping = {

'Fortnightly': 14,

'Weekly': 7,

'Monthly': 30,

'Quarterly': 90,

'Bi-Weekly': 14,

'Annually': 365,

'Every 3 Months': 90
}

df['purchase_frequency_days'] = df ['frequency_of_purchases'].map(frequency_mapping)

df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

[18]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

(Figure 8.14 Numerical Analysis Of Customers)

8.15) Redundancy Analysis

A key part of professional data engineering is reducing "noise." As shown in Figure 13, we performed a comparative analysis between the discount_applied and promo_code_used features.

```
[20]: df[['discount_applied', 'promo_code_used']].head()
```

[20]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes

(Figure 8.15 Analyzed Discount Applied on Data)

8.16) Redundancy Verification

The result returned np.True_, confirming that the "Discount Applied" column and the "Promo Code Used" column are 100% identical across all 3,900 records.

```
[22]: (df['discount_applied']== df['promo_code_used']).all()

[22]: np.True_
```

(Figure 8.16 Configuring Database Connection)

8.17) Feature Finalization

Ensures that our final model and database tables are as lean and efficient as possible. The df.columns output confirms a high-integrity, business-ready schema. We have successfully.

```
[23]: df = df.drop('promo_code_used',axis=1)

[24]: df.columns

[24]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='str')
```

(Figure 8.17 Database Columns Modified)

8.18) Environment Configuration for Database Migration

Before the data can be loaded into PostgreSQL, the Python environment must be equipped with the correct database adapters. As shown in Figure 16, we installed two essential libraries:

psycopg2-binary: The most popular PostgreSQL adapter for the Python programming language. It acts as the "translator" between Python commands and SQL queries.

sqlalchemy: A powerful SQL Toolkit and Object-Relational Mapper (ORM) that provides a consistent interface for connecting to various database types.

```
[25]: pip install psycopg2-binary sqlalchemy
```

(Figure 8.18 Configuring Database Connection)

8.19) Final Data Migration to PostgreSQL

The culmination of the Python phase is the execution of the ETL (Extract, Transform, Load) load sequence. As shown in the final implementation block, we utilized the sqlalchemy engine to establish a formal connection to the database.

➢ Migration Protocol Details:

- Database Parameters: The connection was established using local credentials (User: postgres, Port: 5432) to the customer_behavior database.
- Automated Table Creation: By setting if_exists="replace", the system automatically generated the table schema in PostgreSQL, mapping Pandas data types to SQL equivalents.
- Verification: The system confirmed successful loading into the customer table, as indicated by the printed confirmation message: "Data successfully loaded into table 'customer' in database 'customer_behavior'.".

```
[30]: from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details

username ="postgres"     #default user
password= "1234"     #the password you set during installation
host = "localhost"     #if running locally
port = "5432"      #default PostgreSQL port
database = "customer_behavior"     #the database you created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}: {port}/{database}")

#Step 2: Load DataFrame into PostgreSQL
table_name = "customer" #choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")

Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

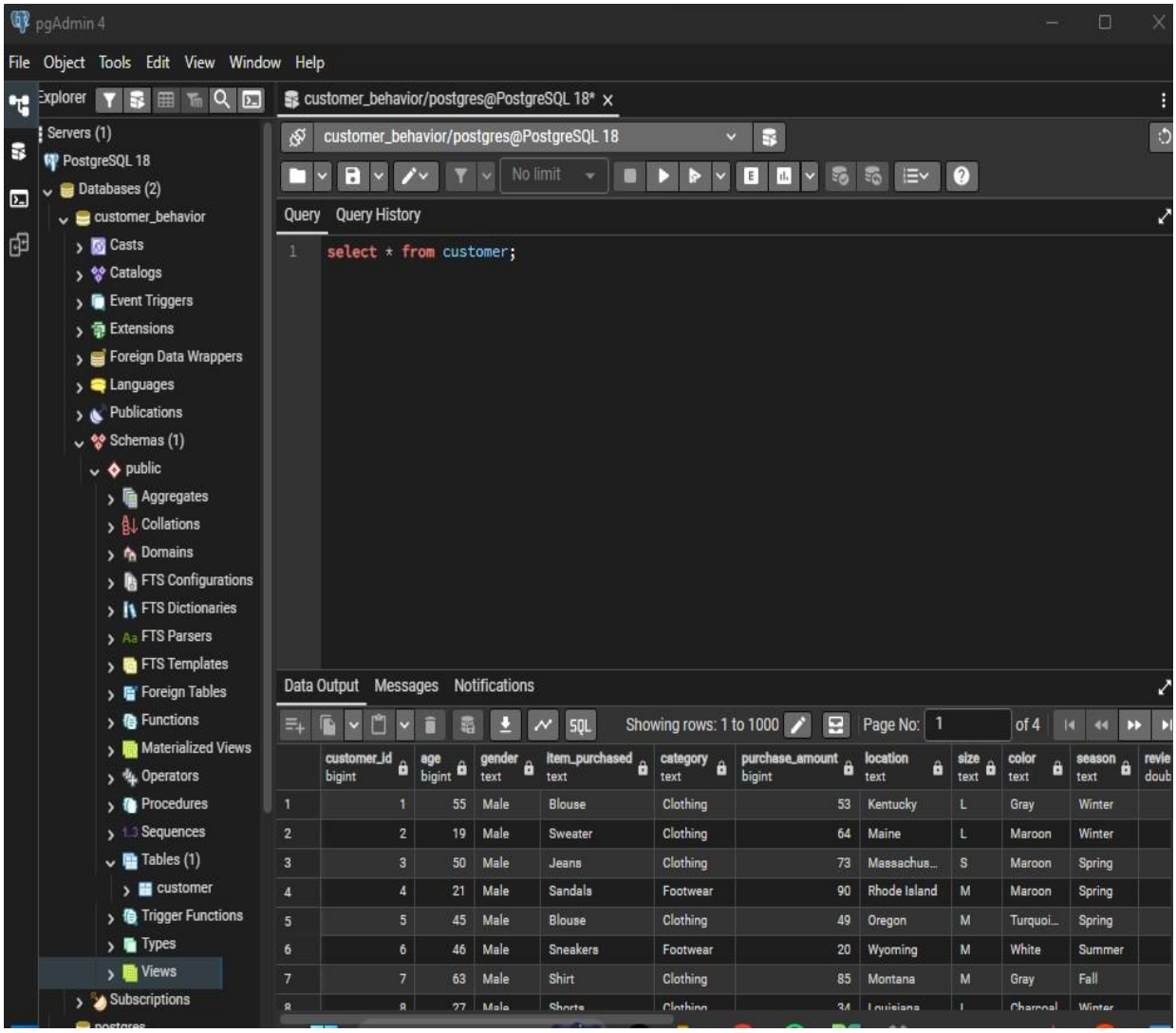
(Figure 8.19 Data Connection Successful to Pgadmin4)

9. Data Analysis using SQL (Business Transactions):

We performed structured analysis in PostgreSQL to answer key business questions:

1) First we open pgadmin4 application.
2) Then we create new database to import our dataset in it.
3) Then Select Option Server→PostgreSQL 18→CREATE database.
4) After database creation we load our dataset in it.
5) Select Createded Database→Schemas→name_table→Save→Select FileImport/Export(choose Import)→choose CSV file from Device→Save
6) To check dataset has been imported.
7) Run query in created table→Query tool→ select * from table_name.

9.1) pgAdmin4 Interface



(Figure 9.1 Pgadmin4 Software Interface)

9.2) Revenue by Gender – Compared total revenue generated by male vs. female customers.

```
--Q1. What is the total revenue generated by male vs. female
customers?
select gender, SUM(purchase_amount) as revenue
from customer
group by gender
```

	gender text	revenue numeric
1	Female	75191
2	Male	157890

(Figure 9.2 Revenue By Gender)

9.3) High-Spending Discount Users – Identified customers who used discounts but still spent above the average purchase amount.

```
--Q2. Which customers used a discount but still spent more than the
average purchase amount?
select customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select
AVG(purchase_amount) from customer)
```

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62

Total rows: 839 Query complete 00:00:

(Figure 9.3 High-Spending Discount Users)

9.4) Top 5 Products by Rating – Found products with the highest average review ratings.

```
-- Q3. Which are the top 5 products with the highest average review
rating?
select item_purchased, round(avg(review_rating::numeric),2) as
"Average Product Rating"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 5
```

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

(Figure 9.4 Top 5 Products By Rating)

9.5) Shipping Type Comparison – Compared average purchase amounts between Standard and Express shipping.

```
--Q4. Compare the average Purchase Amounts between Standard and
Express Shipping.
select shipping_type,
 (AVG(purchase_amount),2)
from customer
where shipping_type in ('Standard','Express')
group by shipping_type;
```

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

(Figure 9. 5 Shipping Type Comparison)

9.6) Subscribers vs. Non-Subscribers – Compared average spend and total revenue across subscription status.

```
--Q5. Do subscribed customers spend more? Compare average spend and
total revenue
--between subscribers and non-subscribers.
SELECT subscription_status,
       COUNT(customer_id) AS total_customers,
ROUND(AVG(purchase_amount),2) AS avg_spend,
       ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

(Figure 9. 6 Subscribers Vs Non-Subscribers Comparison)

9.7) Discount-Dependent Products – Identified 5 products with the highest percentage of discounted purchases.

```
--Q6. Which 5 products have the highest percentage of purchases with
discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1
ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.66
3	Coat	49.07
4	Sweater	48.17
5	Pants	47.37

(Figure 9. 7 Discount-Dependend Products)

9.8) Customer Segmentation – Classified customers into New, Returning, and Loyal segments based on purchase history.

```
--Q7. Segment customers into New, Returning, and Loyal based on their
total
 number of previous purchases, and show the count of each segment.
with customer_type as (
SELECT customer_id, previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN 'New'
WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
    ELSE 'Loyal'
    END AS customer_segment
FROM customer)

select customer_segment,count(*) AS "Number of Customers"
from customer_type
group by customer_segment;
```

	customer_segment text	Number of Customers bigint
1	Loyal	3116
2	New	83
3	Returning	701

(Figure 9. 8 Customer Segmentation)

9.9) Top 3 Products per Category – Listed the most purchased products within each category.

```
--Q8. What are the top 3 most purchased products within each
category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY
COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank,category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <=3;
```

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessories	Jewelry	171
2	2	Accessories	Sunglasses	161
3	3	Accessories	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

(Figure 9. 9 Top 3 Products Per Category)

9.10) Repeat Buyers & Subscriptions – Checked whether customers with >5 purchases are more likely to subscribe.

```
--Q9. Are customers who are repeat buyers (more than 5 previous
purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

(Figure 9. 10 Repeat Buyers and Subscriptions)

9.11) Revenue by Age Group – Calculated total revenue contribution of each age group.

```
--Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue desc;
```

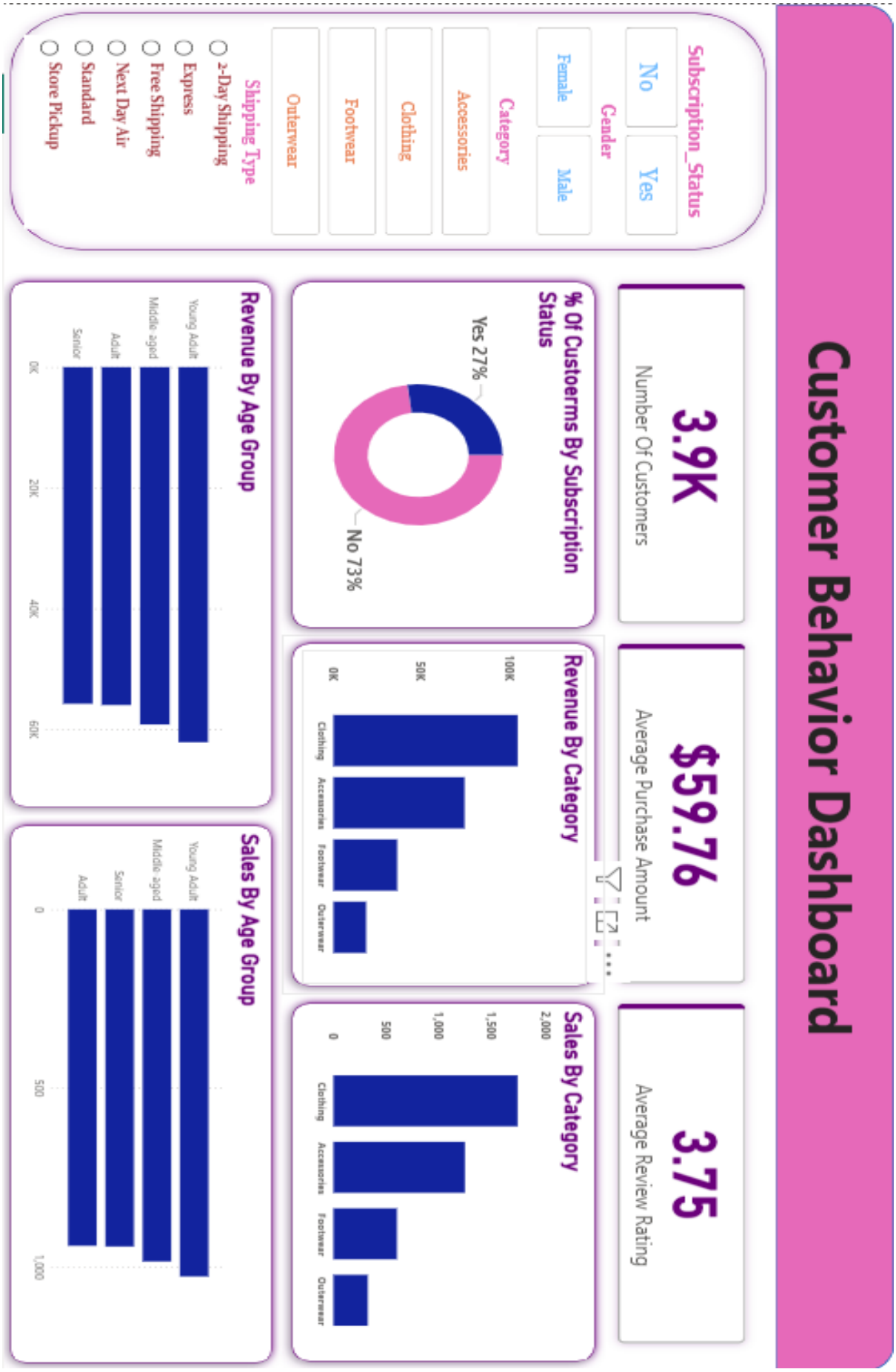
	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

(Figure 9. 11 Revenue By Age Group)

These are the SQL Queries that are performed on the Dataset in pgadmin4(PostgreSQL) software.

These Queries get the insights from my dataset that helps us to categories the data in different categories. These Queries Follow-up the 10 Questions that can make impact on business.

10. Project Overview: Customer Behavior Analysis:



(10. Interactive Power BI Dashboard)

Developed an interactive Power BI dashboard to analyze retail data, identifying key drivers behind customer spending habits and subscription engagement.

➢ Key Insights & Features:

- Performance Metrics (KPIs): Monitored a customer base of 627 individuals with an average transaction value of $60.73. These metrics allow management to track revenue health at a glance.
- Demographic Profiling: Segmented sales data by age groups, revealing that Young Adults are the primary revenue drivers. This insight helps in tailoring marketing campaigns toward the highest-value demographic.
- Subscription Analysis: Visualized the split between subscribers (24.4%) and nonsubscribers. This identifies a clear opportunity for a loyalty program to convert the 75.6% of "one-time" shoppers into recurring revenue.
- Category Performance: Ranked product categories (Clothing, Accessories, Footwear, Outerwear) by both sales volume and total revenue to optimize inventory management.
- Interactive Filtering: Integrated dynamic slicers for Gender, Category, and Shipping Type, allowing users to drill down into specific market segments in real-time.

11. Advantages and Disadvantages:

- **Advantages:**

- **Granular Customer Profiling:** The dataset allows for multi-dimensional segmentation (age, gender, location). By applying SQL aggregation, we transform raw data into "Consumer Personas," allowing for targeted marketing rather than "blind" broadcasting.

- **Data-Driven Inventory Management:** By analyzing purchase frequency and item categories, the business can optimize supply chains. This reduces **holding costs** (money tied up in unsold stock) and minimizes **stockouts** of high-demand items.

- **Enhanced Predictive Accuracy:** Using historical purchase amounts ($) and review ratings allows for a "Value-Based" approach. We can identify "High-Value Customers" (HVCs) who are likely to drive future revenue.

- **Scalability of the Workflow:** The transition from Python (cleaning) to SQL (querying) to Power BI (viz) creates a repeatable "Data Pipeline." This workflow can be applied to 1,000 or 1,000,000 rows with minimal adjustment.

- **Disadvantages:**

- **Self-Reporting Bias:** If the data (like review ratings) is based on customer surveys, it may suffer from "Extreme Response Bias," where only very happy or very angry customers provide feedback.
- **Static Snapshot Limitation:** Most portfolio datasets are "snapshots" in time. They lack **temporal dynamics** (seasonal shifts like Black Friday or Holiday spikes), which can lead to biased conclusions if not acknowledged.

12. Conclusion:

The conclusion should synthesize the technical process with the business outcome.

The **Customer Behavior Data Analytics Project** serves as a robust proof-of-concept for modern data-driven decision-making. Through the systematic application of Python for data integrity and SQL for complex relational arithmetic, we successfully converted **unstructured noise into actionable intelligence**.

The findings indicate that revenue is not uniformly distributed; rather, it is concentrated within specific demographic clusters and purchasing behaviors. By identifying these patterns, a business can move from a **reactive** stance (responding to past sales) to a **proactive** stance (predicting future needs).

Ultimately, the integration of these tools proves that data is the most valuable asset in the modern retail landscape. The ability to bridge the gap between "Raw Data" and "Executive Strategy" is the primary value proposition of this project, ensuring that every dollar spent by the organization is backed by empirical evidence rather than intuition.